

Variables d'environnement - JobbingTrack

Configuration par variables d'environnement

JobbingTrack utilise exclusivement des variables d'environnement pour la configuration. Aucune valeur n'est hardcodée dans les fichiers de configuration.

Variables d'environnement principales

Base de données PostgreSQL

```
POSTGRES_DB=jobbingtrack          # Nom de la base de données
POSTGRES_USER=jobbingtrack        # Utilisateur PostgreSQL
POSTGRES_PASSWORD=jobbingtrack123 # Mot de passe PostgreSQL
```

Frontend

```
NEXT_PUBLIC_API_URL=http://localhost:3000      # URL de l'API Gateway
NEXT_PUBLIC_AUTH_SERVICE_URL=http://localhost:3001 # URL du service d'authentification
NEXT_PUBLIC_METRICS_URL=http://localhost:3014    # URL du service de métriques
```

JWT

```
JWT_SECRET=votre-secret-jwt-unique-et-securise # Secret JWT (générez un secret)
JWT_REFRESH_SECRET=votre-secret-refresh-unique  # Secret refresh token
```

Redis

```
REDIS_URL=redis://localhost:6379 # URL Redis pour le cache
```

Email (SMTP)

```
SMTP_HOST=smtp.gmail.com           # Serveur SMTP
SMTP_PORT=587                      # Port SMTP
SMTP_USER=votre-email@gmail.com    # Email expéditeur
SMTP_PASS=votre-mot-de-passe-app   # Mot de passe application
SMTP_FROM=JobbingTrack <noreply@jobbingtrack.com> # Email expéditeur formaté
```

Utilisateur Administrateur

```
ADMIN_EMAIL=pavel@jobbingtrack.com # Email de l'administrateur
ADMIN_PASSWORD=password123         # Mot de passe de l'administrateur
ADMIN_FIRST_NAME=Pavel             # Prénom de l'administrateur
ADMIN_LAST_NAME=Delhomme           # Nom de l'administrateur
```

Configuration Docker Compose

Variables d'environnement dans docker-compose.yml

```
services:
  postgres:
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

  api-gateway:
    environment:
      - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
      - JWT_SECRET=${JWT_SECRET}
      - JWT_REFRESH_SECRET=${JWT_REFRESH_SECRET}
      - REDIS_URL=redis://redis:6379
```

Variables d'environnement dans backend services

```
environment:
  - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432
  - JWT_SECRET=${JWT_SECRET}
  - AUTH_SERVICE_URL=http://auth-service:3001
```

Configuration de développement

Créez un fichier `.env` à la racine du projet :

```
# Copier depuis .env.example et ajuster
cp .env.example .env

# Éditer le fichier .env
nano .env
```

Variables d'environnement pour le développement

```
# PostgreSQL
POSTGRES_DB=jobbingtrack
POSTGRES_USER=jobbingtrack
POSTGRES_PASSWORD=jobbingtrack123

# JWT
JWT_SECRET=dev-secret-key-2025-change-in-production
JWT_REFRESH_SECRET=dev-refresh-secret-2025

# Utilisateur administrateur
ADMIN_EMAIL=pavel@jobbingtrack.com
ADMIN_PASSWORD=password123
ADMIN_FIRST_NAME=Pavel
ADMIN_LAST_NAME=Delhomme

# Configuration
NODE_ENV=development
LOG_LEVEL=info
```

Configuration de production

Créez un fichier `.env.production` :

```
# PostgreSQL
POSTGRES_DB=jobbingtrack_prod
POSTGRES_USER=jobbingtrack_prod
POSTGRES_PASSWORD=super_secret_password_2025!

# JWT
JWT_SECRET=prod_super_secret_jwt_key_2025_unique_and_secure
```

```
JWT_REFRESH_SECRET=prod_super_secret_refresh_key_2025
```

```
# Utilisateur administrateur  
ADMIN_EMAIL=admin@votredomaine.com  
ADMIN_PASSWORD=votre_password_admin_securise_2025!  
ADMIN_FIRST_NAME=Admin  
ADMIN_LAST_NAME=JobbingTrack  
  
# Configuration  
NODE_ENV=production  
LOG_LEVEL=warn
```

Variables d'environnement spécifiques aux services

Services backend

Tous les services backend utilisent les mêmes variables PostgreSQL mais avec des schémas différents :

- **auth-service** : ?schema=public
- **metrics-aggregator-service** : ?schema=metrics
- **deployment-service** : ?schema=deployment
- **security-service** : ?schema=security

Services frontend

```
NEXT_PUBLIC_API_URL=https://api.votredomaine.com  
NEXT_PUBLIC_AUTH_SERVICE_URL=https://auth.votredomaine.com  
NEXT_PUBLIC_METRICS_URL=https://metrics.votredomaine.com
```

Scripts de génération de données

Les scripts utilisent les variables d'environnement :

```
// Exemple dans generate-test-data.js  
const dbUrl = process.env.DATABASE_URL || 'postgresql://jobbingtrack:jobbingtrack1:
```

Sécurité

⚠ Important - En production :

1. **Générez des secrets uniques** pour JWT
2. **Utilisez des mots de passe forts** pour PostgreSQL
3. **Ne commitez jamais** les fichiers .env
4. **Utilisez des variables d'environnement** dans tous les déploiements

Génération de secrets sécurisés :

```
# Générer un secret JWT
openssl rand -base64 64

# Générer un mot de passe PostgreSQL
openssl rand -base64 32
```

Utilisation

Démarrage avec variables d'environnement

```
# Charger les variables d'environnement
export $(cat .env | xargs)

# Ou utiliser un fichier .env avec docker-compose
docker-compose --env-file .env up

# Ou avec make
make up-full # Utilise automatiquement les variables d'environnement
```

Vérification des variables

```
# Vérifier que les variables sont chargées
env | grep POSTGRES
env | grep JWT
```

Migration des configurations existantes

Si vous avez des configurations hardcodées, migrez vers les variables d'environnement :

✗ Avant (hardcodé)

environment:

- DATABASE_URL=postgresql://admin:admin123@postgres:5432/jobbingtrack
- JWT_SECRET=mon-secret-hardcode

✓ Après (variables d'environnement)

environment:

- DATABASE_URL=postgresql://\${POSTGRES_USER}:\${POSTGRES_PASSWORD}@postgres:5432/!
- JWT_SECRET=\${JWT_SECRET}

Avec les variables d'environnement correspondantes dans `.env` :

```
POSTGRES_USER=jobbingtrack
POSTGRES_PASSWORD=jobbingtrack123
POSTGRES_DB=jobbingtrack
JWT_SECRET=mon-secret-unique-et-securise
```